

Simplified Explanation of MongoDB in This Lab

What MongoDB Is

MongoDB is a **document-oriented NoSQL database**.

Instead of using tables and fixed schemas like relational databases, it stores data as **flexible JSON-like documents** (BSON).

This flexibility makes it ideal for systems where:

- data does not follow one strict structure
 - records include nested fields or change over time
 - telemetry, logs, and model outputs need to be stored together
-

MongoDB's Role in the Adaptive Code Safety Harness

In this lab, MongoDB acts as the **persistent audit store** for every sandbox execution.

Each run can produce many types of information:

- static analysis results
- telemetry (CPU, network, filesystem...)
- sandbox classifications
- model reasoning from Ollama
- risk scores and policy decisions

MongoDB stores all these signals in a **single document per run**, tied to a `run_id`.

Because everything is preserved, investigators can later:

- reconstruct why a decision was made
 - compare past runs
 - check if policies were applied properly
-

Why This Helps Explainability

Ollama produces reasoning about risks and suspicious activity.

MongoDB stores:

- the raw telemetry
- the model's explanation
- confidence values
- triggered risk indicators
- final classification or score

By keeping **evidence + explanation together**, the system becomes transparent and auditable.

How MongoDB Works (Simplified)

Documents and Collections

A database contains **collections**, and each collection contains **documents** (key–value objects).

Example document:

JSON

```
{
  "_id": "run_01",
  "api_id": "api07",
  "classification": "benign",
  "cpu_usage": 18.3,
  "network_events": 0,
  "timestamp": "2026-03-15T10:02:11Z"
}
```

Show more lines

MongoDB allows documents in the same collection to have different fields, which is helpful for variable telemetry data.

Basic MongoDB Operations

Create a Collection

JavaScript

```
use lab10_audit
```

```
db.createCollection("sandbox_runs")
```

Show more lines

Insert Documents

Single:

```
JavaScript
db.sandbox_runs.insertOne({
  run_id: "run_001",
  api_id: "api03",
  classification: "suspicious",
  telemetry: { cpu: 72.5, network_attempts: 3 },
  timestamp: new Date()
})
..
```

Show more lines

Many:

```
JavaScript
db.sandbox_runs.insertMany([
  { run_id: "run_002", classification: "benign" },
  { run_id: "run_003", classification: "potentially_harmful" }
])
```

Show more lines

Query Documents

Return everything:

```
JavaScript
db.sandbox_runs.find()
Show more lines
```

Filter:

```
JavaScript
db.sandbox_runs.find({ classification: "benign" })
```

Show more lines

Advanced:

JavaScript

```
db.sandbox_runs.find({  
  cpu_usage: { $gt: 80 },  
  potentially_harmful: true  
})
```

Show more lines

Projection:

JavaScript

```
db.sandbox_runs.find(  
  { classification: "suspicious" },  
  { api_id: 1, cpu_usage: 1 }  
)
```

Show more lines

Update Documents

JavaScript

```
db.sandbox_runs.updateOne(  
  { run_id: "run_001" },  
  { $set: { classification: "benign" } }  
)
```

Show more lines

Increment fields:

JavaScript

```
db.sandbox_runs.updateOne(  
  { run_id: "run_002" },  
  { $inc: { network_attempts: 1 } }  
)
```

Show more lines

Delete Documents

JavaScript

```
db.sandbox_runs.deleteOne({ run_id: "run_003" })
```

```
db.sandbox_runs.deleteMany({ classification: "benign" })
```

Show more lines

Why MongoDB + Ollama Matters

Together, they provide:

- **Interpretation** (Ollama)
- **Traceability and transparency** (MongoDB)

This combination creates a monitoring pipeline that is:

- flexible
- auditable
- explainable
- easy to analyse later